

Especificación de la Arquitectura de Microservicios para Plataforma E-commerce

1. Introducción

El presente documento técnico describe la arquitectura de microservicios propuesta para la plataforma de comercio electrónico de periféricos de PC. Esta arquitectura se ha seleccionado para proporcionar escalabilidad, resiliencia, flexibilidad y una mayor agilidad en el desarrollo y despliegue de nuevas funcionalidades. Se detallarán los componentes clave, los patrones de comunicación y la infraestructura tecnológica necesaria para soportar este modelo distribuido.

El objetivo es establecer una comprensión clara de cómo los diferentes servicios interactuarán entre sí y con los usuarios finales, así como las consideraciones de diseño que permitirán a la plataforma evolucionar y adaptarse a las futuras demandas del mercado.

2. Componentes Clave de la Arquitectura

2.1. API Gateway

El **API Gateway** es el punto de entrada único para todos los clientes (web, móvil) que interactúan con la plataforma. Su rol principal es actuar como un proxy inverso, enrutando las solicitudes a los microservicios apropiados. Además, el API Gateway centraliza funcionalidades transversales que de otro modo tendrían que implementarse en cada microservicio, reduciendo la complejidad y mejorando la consistencia.

Funcionalidades Clave del API Gateway:

- **Enrutamiento de Solicitudes:** Dirige las solicitudes entrantes al microservicio correcto basándose en reglas predefinidas.
- **Autenticación y Autorización:** Valida las credenciales del usuario y los permisos antes de reenviar la solicitud a un microservicio interno. Esto descarga a los microservicios individuales de esta responsabilidad.
- **Terminación SSL/TLS:** Maneja el cifrado y descifrado de las comunicaciones con los clientes.
- **Limitación de Tasa (Rate Limiting):** Protege los microservicios de sobrecargas controlando el número de solicitudes que un cliente puede realizar en un período de tiempo.
- **Transformación de Solicitudes/Respuestas:** Puede modificar las solicitudes o respuestas para adaptarlas a las necesidades de los clientes o de los microservicios internos.
- **Agregación de Respuestas:** Combina las respuestas de múltiples microservicios en una única respuesta para el cliente, simplificando la lógica del cliente.

2.2. Estrategia de Comunicación entre Servicios

La comunicación entre microservicios puede ser síncrona o asíncrona, y la elección depende de los requisitos específicos de cada interacción. Una arquitectura robusta suele emplear una combinación de ambas.

2.2.1. Comunicación Síncrona (Request/Response)

Se utiliza para operaciones que requieren una respuesta inmediata y donde el servicio solicitante necesita el resultado para continuar su ejecución. Típicamente implementada mediante llamadas HTTP/REST.

- **Uso:** Consultas de datos (ej. el Servicio de Carrito consulta al Servicio de Catálogo para obtener detalles de un producto), operaciones que deben ser completadas antes de que el usuario pueda proceder (ej. autenticación).
- **Tecnologías:** HTTP/REST, gRPC.
- **Consideraciones:** Introduce acoplamiento temporal entre servicios y puede aumentar la latencia. Requiere mecanismos de manejo de fallos (circuit breakers, reintentos) para mejorar la resiliencia.

2.2.2. Comunicación Asíncrona (Event-Driven)

Se utiliza para operaciones que no requieren una respuesta inmediata o que pueden procesarse en segundo plano. Los servicios se comunican mediante eventos a través de un **Message Broker** (ej. Apache Kafka, RabbitMQ).

- **Uso:** Actualizaciones de estado (ej. el Servicio de Pedidos publica un evento de “Pedido Creado” que es consumido por el Servicio de Inventario para reducir el stock y por el Servicio de Notificaciones para enviar un email), tareas de larga duración, desacoplamiento de servicios.
- **Tecnologías:** Message Brokers (Kafka, RabbitMQ), colas de mensajes (SQS).
- **Consideraciones:** Reduce el acoplamiento temporal y espacial, mejora la escalabilidad y la resiliencia. Introduce el desafío de la consistencia eventual y la trazabilidad de eventos.

2.3. Rol del Service Discovery

En una arquitectura de microservicios, los servicios se despliegan y escalan dinámicamente, lo que significa que sus ubicaciones de red (direcciones IP y puertos) pueden cambiar. El **Service Discovery** es un componente crítico que permite a los microservicios encontrarse y comunicarse entre sí sin necesidad de conocer sus ubicaciones físicas.

- **Registro de Servicios:** Cada microservicio se registra a sí mismo en un **Service Registry** al iniciar, proporcionando su ubicación y metadatos.
- **Descubrimiento de Servicios:** Los microservicios (o el API Gateway) consultan el Service Registry para obtener la ubicación de un servicio específico antes de realizar una llamada.
- **Tecnologías:** Eureka, Consul, ZooKeeper, Kubernetes DNS.
- **Beneficios:** Facilita la escalabilidad horizontal, la resiliencia (al permitir que los servicios fallen y se reemplacen sin reconfiguración manual) y la gestión dinámica del entorno.

3. Esquema Detallado de los Componentes de la Infraestructura Tecnológica

La siguiente sección presenta un diagrama D2 que ilustra la disposición de los componentes clave de la infraestructura tecnológica, mostrando las interacciones entre los clientes, el API Gateway, los microservicios, el Service Discovery, el Message Broker y las bases de datos.

Especificación de la Arquitectura de Microservicios para Plataforma E-commerce

1. Introducción

El presente documento describe la arquitectura de microservicios propuesta para la plataforma de comercio electrónico de comercio en línea. Esta arquitectura se ha desarrollado para proporcionar escalabilidad, resiliencia, flexibilidad y una mayor agilidad en el desarrollo y despliegue de nuevas funcionalidades. Se detallan los componentes clave, los patrones de comunicación y la infraestructura tecnológica necesaria para soportar este modelo de diseño.

El objetivo principal de esta especificación es definir la arquitectura de la plataforma tecnológica, considerando tanto la arquitectura de software como la de hardware, para garantizar la escalabilidad y la disponibilidad de la plataforma en todo momento.

2. Componentes Clave de la Arquitectura

2.1 API Gateway

El API Gateway es el punto de entrada único para todos los clientes (web, móvil) que interactúan con la plataforma. Su principal función es actuar como un proxy inverso, enrutando las solicitudes a los microservicios apropiados. Además, el API Gateway proporciona funcionalidades avanzadas que van más allá de la simple intermediación, como la autenticación, la autorización, la limitación de tasa y el manejo de errores.

Funcionalidades Clave del API Gateway:

- Enrutamiento de Solicitudes:** Dirige las solicitudes entrantes al microservicio correcto basándose en reglas predefinidas.
- Autenticación y Autorización:** Valida las credenciales del usuario y los permisos antes de permitir el acceso a los recursos.
- Transformación de Datos:** Modifica el formato de las solicitudes y respuestas para adaptarse a los requisitos de los clientes.
- Limitador de Tasa (Rate Limiting):** Protege los recursos de la sobrecarga controlando el número de solicitudes que pueden realizarse en un período de tiempo.
- Transformación de Solicitudes/Respuestas:** Permite modificar las solicitudes o respuestas para adaptarse a los requisitos de los clientes o de los microservicios internos.
- Aggregación de Respuestas:** Combina las respuestas de múltiples microservicios en una única respuesta para el cliente, simplificando la lógica del cliente.

2.2 Estrategia de Comunicación entre Servicios

La comunicación entre microservicios puede ser sincrónica o asincrónica, y la elección depende de los requisitos específicos de cada funcionalidad. Una arquitectura híbrida puede emplear una combinación de ambos.

2.2.1 Comunicación Sincrónica (Request/Response)

Se utiliza para operaciones que requieren una respuesta inmediata y donde el servicio cliente necesita el resultado para continuar su ejecución. Usualmente implementado mediante llamadas HTTP/REST.

- Uso:** Consultas de datos (ej. el Servicio de Catálogo al Servicio de Catálogo) para obtener información en tiempo real, operaciones que deben ser completadas antes de que el usuario pueda proceder (ej. autenticación).
- Tecnologías:** HTTP/REST, gRPC.
- Consideraciones:** Introduce latencia al depender de la red y puede aumentar la complejidad de los servicios al tener que manejar errores de red.

2.2.2 Comunicación Asincrónica (Event-Driven)

Se utiliza para operaciones que no requieren una respuesta inmediata o que pueden procesarse en segundo plano. Los servicios se comunican mediante eventos a través de un **Message Broker** (ej. Apache Kafka, RabbitMQ).

- Uso:** Actualización de estado (ej. el Servicio de Pedidos publica un evento de "Pedido Creado") que es consumido por el Servicio de Inventario para actualizar el stock y por el Servicio de Recomendación para ofrecer sugerencias.
- Tecnologías:** Message Broker (Kafka, RabbitMQ), Colas de Mensajes (JMS).
- Consideraciones:** Reduce el acoplamiento temporal entre servicios y puede aumentar la latencia. Requiere un diseño de eventos claro y la capacidad de manejar eventos.

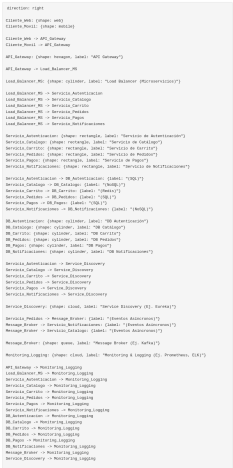
2.3 Rol del Service Discovery

En una arquitectura de microservicios, los servicios se despliegan y escalan dinámicamente, lo que significa que sus ubicaciones (IPs y puertos) pueden cambiar. El **Service Discovery** es un componente crítico que permite a los microservicios encontrar y comunicarse entre sí sin necesidad de conocer sus direcciones fijas.

- Registro de Servicios:** Cada microservicio se registra o notifica en un **Service Registry** central, proporcionando su ubicación y metadatos.
- Descubrimiento de Servicios:** Los microservicios (o el API Gateway) consultan el Service Registry para obtener la ubicación de un servicio específico antes de realizar una llamada.
- Balanceo:** Facilita la escalabilidad horizontal al permitir que los servicios liberen y se reemplacen sin necesidad manual y la gestión dinámica del tráfico.

3. Esquema Detallado de los Componentes de la Infraestructura Tecnológica

La siguiente sección presenta un diagrama D2 que ilustra la disposición de los componentes clave de la infraestructura tecnológica, incluyendo las interacciones entre los clientes, el API Gateway, los microservicios, el Service Discovery, el Message Broker y las bases de datos.



4. Conclusión

La arquitectura de microservicios propuesta, con un API Gateway robusto, una estrategia de comunicación híbrida (sincrónica y asincrónica) y un mecanismo de Service Discovery eficiente, proporciona una base sólida para el desarrollo de una plataforma de comercio electrónico escalable, resiliente y adaptable. El uso de componentes como el Message Broker y las bases de datos distribuidas garantiza la disponibilidad y la integridad de los datos, permitiendo una experiencia de usuario superior.

5. Referencias

- Samuel, B. (2020). *Building Microservices: Creating New Cloud-Native Systems*. O'Reilly Media.
- Fowler, M. (2014). *Microservices: A Definition of This New Architectural Term*. Martin Fowler.com.
- Robustio, C. (2018). *Microservices Patterns: With Examples in Java, Spring and Docker*. No Starch Press.
- Cloud Native Computing Foundation.

direction: right

Cliente_Web: {shape: web}

Cliente_Movil: {shape: mobile}

Cliente_Web -> API_Gateway

Cliente_Movil -> API_Gateway

API_Gateway: {shape: hexagon, label: "API Gateway"}

API_Gateway -> Load_Balancer_MS

Load_Balancer_MS: {shape: cylinder, label: "Load Balancer (Microservicios)"}

Load_Balancer_MS -> Servicio_Autenticacion

Load_Balancer_MS -> Servicio_Catalogo

Load_Balancer_MS -> Servicio_Carrito

Load_Balancer_MS -> Servicio_Pedidos

Load_Balancer_MS -> Servicio_Pagos

Load_Balancer_MS -> Servicio_Notificaciones

Servicio_Autenticacion: {shape: rectangle, label: "Servicio de Autenticación"}

Servicio_Catalogo: {shape: rectangle, label: "Servicio de Catálogo"}

Servicio_Carrito: {shape: rectangle, label: "Servicio de Carrito"}

Servicio_Pedidos: {shape: rectangle, label: "Servicio de Pedidos"}

Servicio_Pagos: {shape: rectangle, label: "Servicio de Pagos"}

Servicio_Notificaciones: {shape: rectangle, label: "Servicio de Notificaciones"}

Servicio_Autenticacion -> DB_Autenticacion: {label: "(SQL)"}

Servicio_Catalogo -> DB_Catalogo: {label: "(NoSQL)"}

Servicio_Carrito -> DB_Carrito: {label: "(Redis)"}

Servicio_Pedidos -> DB_Pedidos: {label: "(SQL)"}

Servicio_Pagos -> DB_Pagos: {label: "(SQL)"}

Servicio_Notificaciones -> DB_Notificaciones: {label: "(NoSQL)"}

DB_Autenticacion: {shape: cylinder, label: "DB Autenticación"}

DB_Catalogo: {shape: cylinder, label: "DB Catálogo"}

DB_Carrito: {shape: cylinder, label: "DB Carrito"}

DB_Pedidos: {shape: cylinder, label: "DB Pedidos"}

DB_Pagos: {shape: cylinder, label: "DB Pagos"}

DB_Notificaciones: {shape: cylinder, label: "DB Notificaciones"}

Servicio_Autenticacion -> Service_Discovery

Servicio_Catalogo -> Service_Discovery

Servicio_Carrito -> Service_Discovery

```
Servicio_Pedidos -> Service_Discovery
Servicio_Pagos -> Service_Discovery
Servicio_Notificaciones -> Service_Discovery

Service_Discovery: {shape: cloud, label: "Service Discovery (Ej. Eureka)"}

Servicio_Pedidos -> Message_Broker: {label: "(Eventos Asíncronos)"}
Message_Broker -> Servicio_Notificaciones: {label: "(Eventos Asíncronos)"}
Message_Broker -> Servicio_Catalogo: {label: "(Eventos Asíncronos)"}

Message_Broker: {shape: queue, label: "Message Broker (Ej. Kafka)"}

Monitoring_Logging: {shape: cloud, label: "Monitoring & Logging (Ej. Prometheus, ELK)"}

API_Gateway -> Monitoring_Logging
Load_Balancer_MS -> Monitoring_Logging
Servicio_Autenticacion -> Monitoring_Logging
Servicio_Catalogo -> Monitoring_Logging
Servicio_Carrito -> Monitoring_Logging
Servicio_Pedidos -> Monitoring_Logging
Servicio_Pagos -> Monitoring_Logging
Servicio_Notificaciones -> Monitoring_Logging
DB_Autenticacion -> Monitoring_Logging
DB_Catalogo -> Monitoring_Logging
DB_Carrito -> Monitoring_Logging
DB_Pedidos -> Monitoring_Logging
DB_Pagos -> Monitoring_Logging
DB_Notificaciones -> Monitoring_Logging
Message_Broker -> Monitoring_Logging
Service_Discovery -> Monitoring_Logging
```

4. Conclusión

La arquitectura de microservicios propuesta, con un API Gateway robusto, una estrategia de comunicación híbrida (síncrona y asíncrona) y un mecanismo de Service Discovery eficiente, proporciona una base sólida para el desarrollo de una plataforma de e-commerce escalable, resiliente y mantenible. El esquema de componentes ilustra la interconexión de los diferentes elementos, destacando la independencia de los microservicios y la centralización de las preocupaciones transversales. Esta especificación

servirá como una guía fundamental para el equipo de desarrollo y operaciones, asegurando la implementación de una solución tecnológica de vanguardia.

5. Referencias

- [Newman, S. \(2015\). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.](#)
- [Fowler, M. \(2014\). Microservices: A Definition of This New Architectural Term. MartinFowler.com.](#)
- [Richardson, C. \(2018\). *Microservices Patterns: With examples in Java*. Manning Publications.](#)
- [D2 Language Documentation.](#)